

UNIVERSIDAD NACIONAL DE COLOMBIA — SEDE BOGOTÁ

GYMLY

Sistema de gestión integral para gimnasios pequeños

Walter Julián Fernández González

Alessandro Garzón Melo

Jarol David Zuluaga Grau

Sebastián Olaya Sánchez

Ingeniería de Software 1 • Entrega Final

INDICE

01

Presentación del software

Qué es Gymly, flujo de uso y cumplimiento de requerimientos.

02

Aspectos técnicos

Arquitectura, modelo de datos y el reto de pagos con Mercado Pago.

03

Reflexión final

Qué aprendimos, qué nos faltó y qué haríamos distinto.

QUÉ ES GYMLY

Un sistema de gestión integral para un gimnasio, pensado para el contexto colombiano — precios en COP, zona horaria de Bogotá — con dos audiencias claras.

Clientes

Ven los planes en la landing pública, se registran, se inscriben, pagan (efectivo, transferencia o tarjeta) y consultan su vigencia, historial y rachas de constancia.

Administradores

Validan el acceso en recepción con un semáforo visual, aprueban o rechazan pagos manuales, gestionan miembros y planes, y consultan reportes financieros.

FUNCIONALIDADES Y CUMPLIMIENTO

Principales

- ✓ Registro, login e inscripción a planes
- ✓ Pagos en efectivo, transferencia y tarjeta (Mercado Pago)
- ✓ Control de acceso con semáforo en recepción
- ✓ Panel administrativo: miembros, planes y pagos

Secundarias

- ✓ Descuentos vigentes aplicados por plan
- ✓ Rachas de asistencia y reportes financieros
- ✓ Notificaciones programadas de vencimiento e inactividad
- Envío real de notificaciones – cola lista, falta el canal (email/SMS)

ARQUITECTURA DEL SISTEMA

PROJECT-FRONTEND

React 19 · Vite · TypeScript · Tailwind 4

- pages
- components (landing / dashboard / admin / auth)
- services (capa HTTP)
- contexts / hooks (estado global)

PROJECT-BACKEND

Kotlin 2.2 · Spring Boot 4 · PostgreSQL

- controller
- service
- repository (JPA / Hibernate)
- model + dto

EL RETO TÉCNICO CENTRAL: PAGOS CON MERCADO PAGO

Navegador

confirma al volver de MP

Webhook

notificación de MP en producción

Scheduler

reconcilia cada 20s — red de seguridad en local

Los tres caminos son idempotentes: si el pago local ya no está pendiente, no hacen nada.

EL ERROR QUE NO VOLVEREMOS A COMETER

Al inicio usábamos el id local como referencia externa de Mercado Pago. Pagos aprobados *viejos* de la misma cuenta (otros entornos, bases reiniciadas) coincidían con ids pequeños y aprobaban pagos que debían rechazarse. La solución: un UUID único global como referencia de cada pago.

AUTOAPRENDIZAJE: LO QUE MÁS COSTÓ

Frameworks desde cero

Ninguno del equipo había trabajado con estos stacks; curva de aprendizaje fuerte

Integración con Mercado Pago

Los webhooks no funcionan contra localhost, hubo que diseñar un mecanismo alternativo de confirmación para desarrollo.

Estado global en React

Decidir entre Context API y librerías externas, y mantener la sesión y los datos del panel admin sincronizados.

Conectar frontend y backend

Acordar contratos de API, manejo de errores consistente y autenticación entre dos repositorios independientes.

LO QUE APRENDIMOS COMO EQUIPO

01 Escribir código limpio siguiendo principios SOLID y Clean Code

02 Conectar de verdad un frontend con un backend independiente

03 Hacer commits claros y usar GitHub correctamente en equipo (aún nos falta)

EL CURSO Y NUESTRAS LIMITACIONES

Sobre el curso

Sentimos que el curso fue muy completo, no identificamos temas importantes que hayan faltado en el temario.

Tal vez profundizar un poco más en patrones de diseño, ya que estos son esenciales para entrevistas técnicas hoy en día.

Nuestras limitaciones

- Necesitamos mejor comunicación como equipo
- Revisar con más cuidado lo que sugiere la IA: todavía alucina
- Manejar mejor el tiempo y ser más constantes
- Manejar mejor el tiempo y ser más constantes
- Manejar mejor el tiempo y ser más constantes

GRACIAS

Gymly — Ingeniería de Software 1

Walter Julián Fernández González

Alessandro Garzón Melo

Jarol David Zuluaga Grau

Sebastián Olaya Sánchez